

Continuous Control with Deep Reinforcement Learning for Mobile Robot Navigation

Jiaqi Xiang¹, Qingdong Li¹, Xiwang Dong^{1*}, Zhang Ren²

1. School of Automation Science and Electrical Engineering, Beihang University, Beijing, 100191, P.R. China
E-mail: xwdong@buaa.edu.cn

2. Beijing Advanced Innovation Center for Big Data and Brain Computing, Beihang University, Beijing 100191, P.R. China

Abstract—Autonomous navigation is one of the focuses in the field of mobile robot research. The traditional method usually consists of two parts: building the map of environment, localization of mobile robot and path planning. However, these traditional methods usually rely on high-precision sensor information. At the same time, mobile robots have no intelligent understanding of autonomous navigation. In this article, a deep reinforcement learning method, i.e. soft actor critic, is used to navigate in a mapless environment. It takes laser scanning data and information of the target as input, outputs linear velocity and angular velocity in continuous space. The simulation shows that this learning-based end-to-end autonomous navigation method can accomplish tasks as well as traditional methods.

Keywords—Mobile Robot, Deep Reinforcement Learning, Soft Actor Critic, Autonomous navigation

I. INTRODUCTION

The use of mobile robots has been well popularized, such as delivery and rescue. Being able to navigate autonomously in a complex environment is a prerequisite for completing these tasks. In the traditional method, autonomous navigation can be divided into two parts. The first part is the localization and mapping of mobile robots. Mobile robots rely on various sensors to sense the external environment and their own postures. This process is called Simultaneous Localization and Mapping (SLAM) [1]. It uses the information of laser or camera sensor to build an obstacle map. The second part is path planning. The purpose of path planning is to plan a path from the starting point to the target point without collision. The main methods of path planning include heuristic search methods such as A-star algorithm [2], random sampling-based methods such as Rapidly-exploring Random Tree (RRT) [3] and the methods based control theory such as dynamic window approach [4]. Today, the traditional methods have matured. However, most of these methods are dependent on obstacle map, and the mobile robot does not have an intelligent understanding of the navigation problem. In addition, the traditional method is very dependent on high-precision sensors. Nowadays, there are already many indoor localization methods, such as UWB localization [5]. So the position of the mobile robot and the position of the target point can be got easily. In this case, it is important to propose a mapless navigation method.

Learning-based methods have been successfully applied to autonomous navigation. Imitate learning [6] and supervised learning [7] can get good results. This type of method works by training human-labeled data through deep neural networks.

Its drawback is obvious because it is limited by the amount of data.

Reinforcement learning is different from traditional supervised learning and unsupervised learning. It is a process in which an agent learns by interacting with the environment. It does not require human-labeled data. In 2011, Mohammad proposed using the Q-learning algorithm to solve the problem of robot navigation in an unknown environment [8]. However, limited to the size of the Q-learning state space, this method is not suitable for complex environments. In 2013, Mnih et al. proposed the basic method of deep reinforcement learning, Deep Q-Network (DQN) model [9]. Deep reinforcement learning deals with higher dimensional state spaces by introducing deep neural network to fitting value functions. Deep reinforcement learning is currently used in a variety of robot control applications. Tung-Cheng Wu et al. use depth images as input to complete the autonomous obstacle avoidance of drones [10]. Kendall et al. successfully applies deep reinforcement learning to autonomous driving, outputting continuous steering and speed commands [11]. Y. Zhu et al. proposed a target-driven deep reinforcement learning visual navigation method [12]. All of this work proves the effectiveness of deep reinforcement learning on robot autonomous navigation.

In recent years, lots of methods for deep reinforcement learning have been proposed. The DQN is a value-based method, so it can't satisfy the continuous action space. Lillicrap et al. combined the DQN method with the Actor-Critic policy gradient method and proposed the Deep Deterministic Policy Gradient (DDPG), which extends it to the continuous action space [13]. Policy gradient based methods such as Trust region policy optimization (TRPO) [14] and Proximal Policy Optimization (PPO) [15] can achieve better stability. Soft Actor Critic (SAC) [16] is an algorithm which optimizes a stochastic policy in an off-policy way, forming a bridge between stochastic policy optimization and DDPG-style approaches. It incorporates the clipped double-Q trick. And due to the inherent stochasticity of the policy in SAC, it also winds up benefiting from something like target policy smoothing. Therefore, SAC algorithm was chosen as the training algorithm in this paper. At the same time, the Robot Operating System (ROS) and Gazebo were used to simulate the virtual environment. Experiments show that the mobile robot can quickly learn the autonomous navigation and obstacle avoidance policy.

The main contributions of this paper are as follows:

- An continuous control method for autonomous navigation of mobile robots in mapless environment based on SAC algorithm is proposed.
- Inputs are low-dimensional laser scan data and target information. By this way, it can effectively reduce the resource required for calculation.
- In order to make full use of the previously observed states, actions and rewards, the LSTM [17] neural network is used to fit the value function and Q function.

The remainder of this paper is organized as follows: Section II presents the details of the algorithm. Section III is the description of the simulation experiment and the result. Section IV is a summary and conclusion of this article.

II. MOTION PLANNER IMPLEMENTATION

A system to learning the continuous control was designed so that the mobile robot can reach the target position quickly without collision.

A. Problem description

The goal of this paper is to provide a continuous control strategy for autonomous navigation of mobile robots. This problem can be described by the function:

$$v_t = f(l_t, p_t, a_{t-1}, r_{t-1}) \quad (1)$$

Where l_t is the laser scanning data, p_t is the relative position and angle of target, a_{t-1} is the action in the last time step(the velocity in the last time step), and r_{t-1} is the reward in the last time step. The above four items can be regarded as the state s_t of the mobile robot. The model trained by the reinforcement learning method will fit the function f to control the velocity of the robot.

B. Soft Actor Critic

Soft Actor Critic (SAC) is an off-policy algorithm. It is the first RL algorithm that combines off-policy, actor critic and maximum entropy. SAC algorithm is an entropy-regularization reinforcement learning method.

In entropy regularization reinforcement learning, an entropy term is added to the standard reinforcement learning objective, so that the optimal policy will maximize its entropy at each visited state:

$$\pi^* = \arg \max_{\pi} E_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot|s_t))) \right] \quad (2)$$

Where π^* is the optimal policy. τ is a trajectory sampled in an episode. γ is discounted rate that determines how importance of the future rewards. $R(s_t, a_t, s_{t+1})$ is the reward while choosing action a_t and making state s_t to s_{t+1} . $H(\pi(\cdot|s_t))$ is the entropy of $\pi(\cdot|s_t)$, α is the trade-off coefficient.

Similarly, the state value function is changed to:

$$V^{\pi}(s) = E_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t (R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot|s_t))) \right] | s_0 = s \quad (3)$$

The action state value function is changed to:

$$Q^{\pi}(s, a) = E_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R + \alpha \sum_{t=1}^{\infty} \gamma^t H(\pi(\cdot|s_t)) \right] | s_0 = s, a_0 = a \quad (4)$$

SAC algorithm learning a policy π_{θ} , a state value function V_{ψ} , and two action value functions Q_{ϕ_1} , Q_{ϕ_2} .

For faster and more stable training, the author introduced two Q networks and then selected one with a smaller Q value as the target Q value each time. The two action value functions update the Bellman equation through the target value network. The loss function is as follows:

$$L(\phi_i, D) = E_{(s, a, r, s', d) \sim D} \left[\left(Q_{\phi_i}(s, a) - (r + \gamma(1-d)V_{\psi_{\text{arg}}}(s')) \right)^2 \right] \quad (5)$$

With definitions, $V^{\pi}(s)$ and $Q^{\pi}(s, a)$ are connected by:

$$\begin{aligned} V^{\pi}(s) &= E_{a \sim \pi} [Q^{\pi}(s, a)] + \alpha H(\pi(\cdot|s)) \\ &= E_{a \sim \pi} [Q^{\pi}(s, a) - \alpha \log \pi(a|s)] \\ &\approx Q^{\pi}(s, a) - \alpha \log \pi(a|s) \end{aligned} \quad (6)$$

SAC algorithm uses minimum Q-value between the two approximators while updating the value function, and it does not use the actions in the experience replay buffer, but samples from the current policy:

$$L(\psi, D) = E_{\substack{s \sim D \\ a \sim \pi_{\theta}}} \left[\left(V_{\psi}(s) - (\min_{i=1,2} Q_{\phi_i}(s, a) - \alpha \log \pi_{\theta}(a|s)) \right)^2 \right] \quad (7)$$

Policy should maximize the expected return and entropy. In other words, it needs to maximize $V^{\pi}(s)$. In the SAC algorithm, squashed Gaussian policy is usually adopted, which means that actions are sampled as:

$$a_{\theta}(s, \xi) = \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \odot \xi), \xi \sim N(0, I) \quad (8)$$

The tanh activation function in the SAC policy ensures that the action generated by the policy is within the effective action space.

A trick was used to improve exploration at the start of training. In the first few steps, the agent selects an action from a uniform random distribution. After that, it will sample from the SAC policy.

The pseudocode of the SAC algorithm is as follows:

Algorithm 1 Soft Actor Critic

1. initial policy π_θ , Q function Q_ϕ , V-function V_ψ
2. initial empty replay buffer D
3. **repeat**
4. Observe state s and select action $a \sim \pi_\theta$
5. Execute a in the environment
6. Observe next state s' , reward r , and done d
7. Store (s, a, r, s', d) in buffer
8. If s' is terminal, reset state
9. **if** it's time to update **then**
10. **for** j in range (however many updates) **do**
11. Randomly sample a batch from buffer D
12. Compute targets for Q and V functions:
 $y_q(r, s', d) = r + \gamma(1-d)V_{\psi_{\text{arg}}}(s')$
 $y_v(s) = \min_{i=1,2} Q_{\phi_i}(s, \tilde{a}) - \alpha \log \pi_\theta(\tilde{a} | s), \tilde{a} \sim \pi_\theta(\cdot | s)$
13. Update Q-functions by gradient descent
 $\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,s',r,d) \in B} (Q_{\phi_i}(s,a) - y_q(r, s', d))^2, \text{ for } i=1,2$
14. Update V-function by gradient descent
 $\nabla_{\psi} \frac{1}{|B|} \sum_{s \in B} (V_{\psi}(s) - y_v(s))^2$
15. Update policy by gradient descent
 $\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} (Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s) | s))$
16. Update target value network with
 $\psi_{\text{arg}} \leftarrow \rho \psi_{\text{arg}} + (1-\rho) \psi$
17. **end for**
18. **end if**
19. **until** convergence

C. State Space And Action Space

1) *State Space*: In order to complete the task, the state space was defined as three parts. The first part is 10-dimensional laser scanning data with scanning angle 360 degrees. The second is the angle and distance between robot and target position. The third is the action selected at the last moment and the reward. The laser scanning data can help the robot learn to avoid obstacles, the information of the target point informs the goal of the robot navigation. The action and reward of the previous moment can more effectively make the robot learn the correctness of the current action.

2) *Action Space*: The action space consists of two dimensions, one for linear velocity and one for angular velocity. All of these velocity command is represented by a squashed Gaussian policy. A clip function is used to ensure the linear velocity is limited to $[0, 0.2] \text{ m/s}$, and the angular velocity is limited to $[-1, 1] \text{ m/s}$.

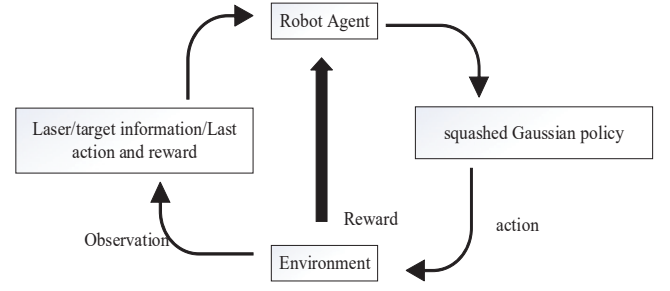


Figure 1. Process of continuous control with deep reinforcement learning for mobile robot navigation.

D. Reward Function

In order to let the agent better understand the goal to learn, that is, to reach the target position as soon as possible without collision. The reward function of this paper is defined as follows: when the mobile robot collides, a negative reward of -100 is given. If the mobile robot reaches target position, a positive reward of +500 is provided.

When the mobile robot moves freely in the environment, the reward depends on the difference from the target position compared with last time step. For reaching the target as fast as possible, a reward of -3 is given for every time step.

Combined with the above four parts, the reward function is defined as:

$$r_t(s_t, a_t) = \begin{cases} 500 & \text{if arrive} \\ -100 & \text{if collided} \\ -3 + k \cdot (d_{t-1} - d_t) & \end{cases} \quad (9)$$

Where k is a hyperparameter.

E. Network Structure

In the SAC algorithm, it is necessary to define three neural network structures for π_θ , V_ψ and Q_ϕ .

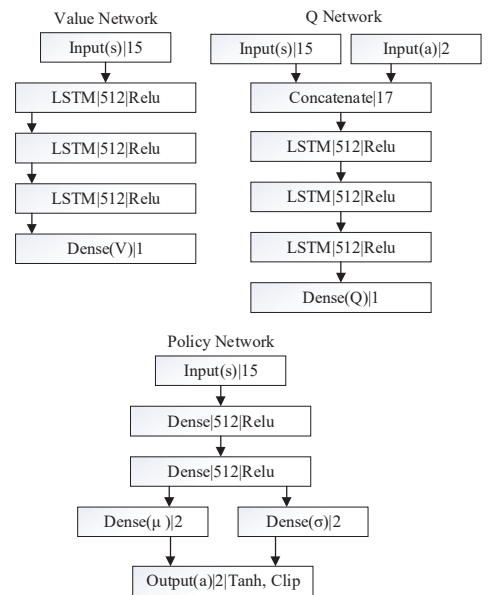


Figure 2. The Network of the SAC model. Each layer includes its type, dimension, and activation function information.

As shown in Fig. 2, the input of value network is a 15-dimensional vector that includes laser scanning data(10-dimensional), target position(2-dimensional, distance and angle), the action of last time step(2-dimensional) and reward(1-dimensional). In order to make the network have the memory of the previous observations and can learn the prediction of the motion, LSTM is chosen as the hidden layer with 512 nodes. After three hidden layers, the output is a state value function V_ψ , which is used to evaluate the quality of the state.

Q network is used to fit the Q value, which evaluate the quality of the action is taken in a certain state. The input consists of a state vector and an action vector, which we merge into a 17-dimensional vector. Similar to the value network, the final output is a one-dimensional Q value.

The policy network is used to generate a Gaussian distribution. After processing the input state vector through two fully connected layers, the mean and standard deviation are generated respectively. Then the action (2-dimensional velocity command) is generated according to the Gaussian distribution. The tanh activation function and clip operation are used to constrain the linear velocity in $[0, 0.2] \text{ m/s}$, and the angular velocity is limited to $[-1, 1] \text{ m/s}$.

III. SIMULATIONS

The training environment was implemented in Gazebo and ROS. An open source mobile robot model Turtlebot3 was used.

A. Training procedure

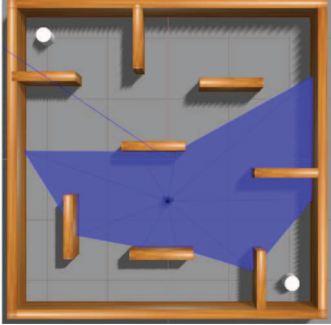


Figure 3. The virtual environment named Turtlebot3_stage_4 in Gazebo.

The virtual environment is shown as Fig. 3. It is a $5 \times 5 \text{ m}^2$ square area with 8 obstacles. The initial position of the mobile robot is $(0, -0.6)$. At each episode, the position of the mobile robot is initialized and the position of the target point are generated randomly.

The hyperparameters for reinforcement learning training are as follows. The learning rates for policy, value and Q network are 0.0003. Adam optimizer is used to perform gradient descent. The size of experience replay buffer is 50000. The maximum episode is 1300. The maximum step for each episode is 1000. The batch size is 128. The discounted rate γ is 0.99.

The model was trained on a PC with Nvidia GeForce RTX 2070 GPU, 3.70 GHz eight core AMD Ryzen 7 2700X processor and 16 GB memory. The environment is speeded up 10 times to train. The training procedure took three hours.

F. Results

The simulation results is shown in Fig. 4. It can be seen that the mobile robot always hits obstacles at the beginning and get a negative reward. With constant trial and error, the mobile robot learned the policy to complete the autonomous navigation task and approached convergence. For comparison, the LSTM layers in the value and Q network were replaced with fully-connected layer while other parameters remained unchanged. It can be seen that the mean reward of LSTM network has increased faster than fully-connected networks. The success rate is shown in Fig. 5. It can be seen that the success rate was increasing with the convergence of policy. Of course, as training time increases, the success rate will become higher and higher.

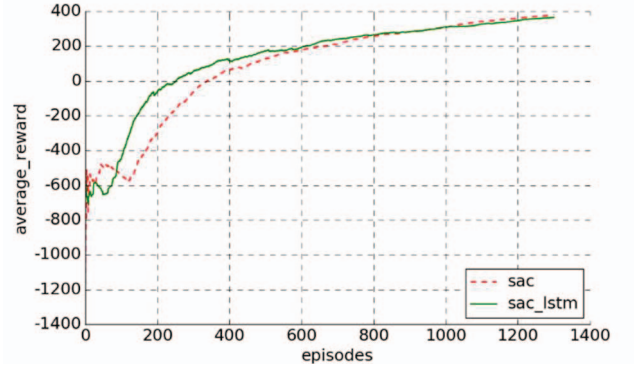


Figure 4. The mean reward over episodes in training procedure. The solid line is the result of using the LSTM network structure, and the dashed line is the result of the fully connected layer network.

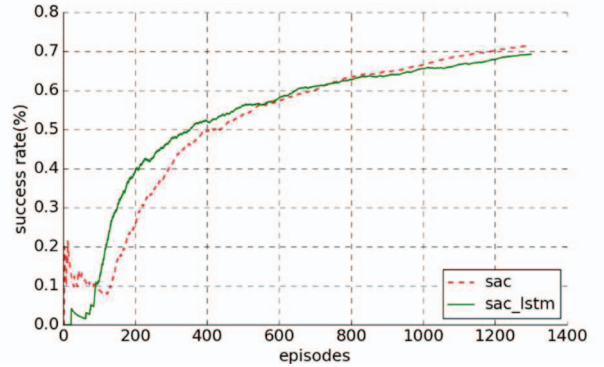


Figure 5. The success rate over episodes in training procedure. The solid line is the result of using the LSTM network structure, and the dashed line is the result of the fully connected layer network.

G. Evaluation

1) *Baseline*: In order to evaluate the effectiveness of the proposed method, traditional methods are implemented for comparison. First, a map of simulation environment was built by the *gmapping* package, which is a package contains a ROS wrapper for *OpenSlam's gmapping*. The package provides laser-based SLAM. The occupancy grid map can be built from laser and odometer data of mobile robots. The occupancy grid map of the virtual environment in this article is as follows.

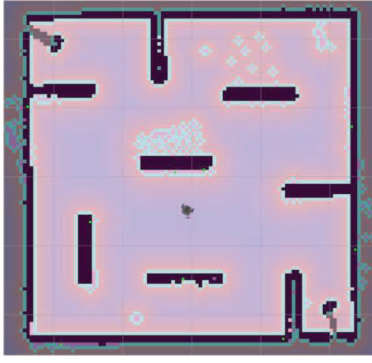


Figure 6. The occupancy grid map of the virtual environment.

The real time position was provided by *amcl* package in ROS. It uses particle filter to track the position of mobile robot in a map. The *move_base* package is used for path planning. It can provide a global and local planner for navigation task by using the A-star algorithm for global path planning and the DWA algorithm for local path planning.

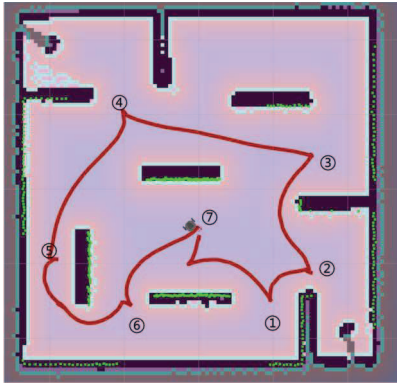


Figure 7. The path planned by *move_base*.

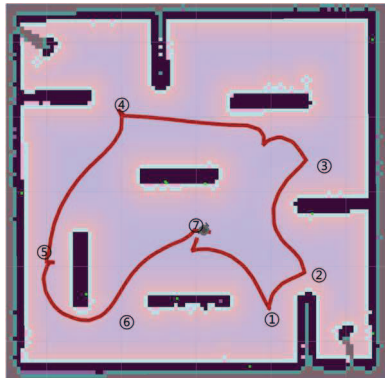


Figure 8. The path planned by reinforcement learning model.

2) *Evaluation*: As shown in Fig. 7 and Fig. 8, seven targets were set for mobile robot. The robot's goal is to arrive these target points in turn. It is worth noting that for *move_base*, building a map is a prerequisite for path planning. But for reinforcement learning methods, maps are only used for visualization of paths. As shown in the figure, both methods can accomplish the task well, and the trajectory of *move_base* planning is better in some places. But it needs to be explained that *move_base* uses 360-dimensional laser scanning data, while the reinforcement learning model uses only 10-dimensional sparse laser scanning data. The result

proves the effectiveness and superiority of the navigation method based on reinforcement learning.

IV. CONCLUSION

In this article, a reinforcement learning method based on Soft Actor Critic was presented for mobile robot navigation in mapless environment. Continuous control of mobile robots was achieved by using 10-dimensional laser scanning data, target information, actions and rewards from the last time step as input. After a few hours of training, the method can achieve similar results to the traditional method. In addition, this continuous control method based on reinforcement learning can accomplish other more complex tasks. Of course, although it has achieved good results in the simulation environment, the application of virtual to real world still needs to be carried out in future work.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China under Grants 61873011 and 61803014, the Beijing Natural Science Foundation under Grant 4182035, the Young Elite Scientists Sponsorship Program by CAST under Grant 2017QNRC001, the Aeronautical Science Foundation of China under Grants 2016ZA51005 and 20170151001, the Special Research Project of Chinese Civil Aircraft, the State Key Laboratory of Intelligent Control and Decision of Complex Systems, and the Fundamental Research Funds for the Central Universities under Grant YWF-18-BJ-Y-73.

REFERENCES

- [1] H. Durrant-Whyte, T. Bailey, "Simultaneous localization and mapping (SLAM)," *IEEE Robotics & Automation Magazine*, vol. 13, no. 2, pp. 99-110, 2006.
- [2] J. Peng, Y. Huang, G. Luo, "Robot path planning based on improved A* Algorithm," *Cybernetics & Information Technologies*, vol. 15, no. 2, pp. 171-180, 2015.
- [3] S. Chakravorty, S. Kumar, "Generalized sampling-based motion planners," *IEEE Transaction on Systems Man & Cybernetics*, vol. 41, no. 3, 2011.
- [4] D. Fox, W. Burgard, S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23-33, 1997.
- [5] S. Krishnan, P. Sharma, Z. Guoping, O. H. Woon, "A UWB based localization system for indoor robot navigation," in *Proceedings of the IEEE International Conference on Ultra-Wideband*, 2007, pp. 77-82.
- [6] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, "Learning monocular reactive UAV control in cluttered natural environments," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2012.
- [7] N. Smolyanskiy, A. Kamenev, J. Smith, S. Birchfield, "Toward low-flying autonomous MAV trail navigation using deep neural networks for environmental awareness," in *Proceedings of the IEEE International Conference on Intelligent Robots and Systems*, 2017.
- [8] M. A. K. Jaradat, M. Al-Rousan, L. Qadan, "Reinforcement based mobile robot navigation in dynamic environment," *Robotics and Computer-Integrated Manufacturing*, vol. 27, no. 1, pp. 135-149, 2011.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [10] T. C. Wu, S. Y. Tseng, C. F. Lai, C. Y. Ho, Y. H. Lai, "Navigating assistance system for quadcopter with deep reinforcement learning," *arXiv preprint arXiv:1811.04584*, 2018.
- [11] A. Kendall, J. Hawke, D. Janz, "Learning to Drive in a Day," *arXiv preprint arXiv:1807.00412*, 2018.

- [12] Y. Zhu, R. Mottaghi, E. Kolve, J. J. Lim, A. Gupta, L. Fei-Fei, A. Farhadi, "Target-driven visual navigation in indoor scenes using deep reinforcement learning," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2017, pp. 3357-3364.
- [13] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, "Continuous control with deep reinforcement learning," *Computer Science*, vol. 8, no. 6, 2015.
- [14] J. Schulman, S. Levine, P. Abbeel, M. Jordan, P. Moritz, "Trust region policy optimization," *International Conference on Machine Learning*, , 2015, pp. 1889-1897.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [16] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, "Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor," *arXiv preprint arXiv:1801.01290*, 2018.
- [17] S. Hochreiter, Schmidhuber, Jürgen, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.